# Endo's Formula Graph and Capability Protocols as Session Types

A Case Study in Object-Capability Protocol Verification

The Prologos Project - from Zachary Larson

2026-03-07

## Contents

**1 Abstract** **4**

**2 1. Endo Architecture Overview** **4**
2.1 1.1 What Is Endo? . . . . . 4
2.2 1.2 Key Participants . . . . . 5
2.3 1.3 Key Data Types . . . . . 5
2.4 1.4 Formula Types . . . . . 6

**3 2. The Formula Graph** **7**
3.1 2.1 Graph Structure . . . . . 7
3.2 2.2 Union-Find Groups . . . . . 8
3.3 2.3 Persistence Invariant . . . . . 9
3.4 2.4 Single Async Mutex . . . . . 9

**4 3. Session Type Notation** **9**

**5 4. CapTP: Capability Transport Protocol** **10**
5.1 4.1 Transport Layers . . . . . 10
5.2 4.2 The CapTP Message Protocol . . . . . 11
5.3 4.3 Session Types for CapTP . . . . . 11
5.4 4.4 Slot Duality . . . . . 13
5.5 4.5 Protocol-Level Observations . . . . . 14

**6 5. Formula Lifecycle Protocol** **14**
6.1 5.1 The Lifecycle State Machine . . . . . 14
6.2 5.2 Session Type for Formula Lifecycle . . . . . 15
6.3 5.3 Critical Ordering: Synchronous Controller Creation . . . . 16
6.4 5.4 Transient Pinning . . . . . 17

**7 6. Agent Protocols** **17**
7.1 6.1 Host Agent . . . . . 17
7.2 6.2 Guest Agent . . . . . 19
7.3 6.3 Capability Attenuation: Host → Guest . . . . . 21

**8 7. Worker Isolation Protocol** **21**

**9 8. Pet Store: Capability Naming Protocol** **22**
9.1 8.1 Core Protocol . . . . . 23
9.2 8.2 Bidirectional Mapping . . . . . 24
9.3 8.3 Protocol-Level Observation: Write Ordering . . . . . 24

**10 9. Mail and Messaging Protocol** **24**
10.1 9.1 Message Types . . . . . 24
10.2 9.2 Mail Protocol . . . . . 25
10.3 9.3 Capability Introduction Pattern . . . . . 26

**11 10. Garbage Collection Protocol** **27**
11.1 10.1 GC Algorithm as Protocol Steps . . . . . 27
11.2 10.2 GC Trigger Protocol . . . . . 28
11.3 10.3 Protocol-Level Observations . . . . . 28

**12 11. Invitation and Peering Protocol** **29**

**13 12. Cancellation and Context Protocol** **30**

**14 13. Connection and Remote Control** **32**

**15 14. Composed Protocol: Full Daemon Bootstrap** **33**

**16 15. Protocol-Level Issues and Recommendations** **34**
16.1 15.1 Mutex Re-Entrancy . . . . . 34
16.2 15.2 Pin Leak Prevention . . . . . 35
16.3 15.3 Promise Resolution GC Ordering . . . . . 35
16.4 15.4 $\mathrm{CTP}_{\mathrm{DROP}}$ Before $\mathrm{CTP}_{\mathrm{DISCONNECT}}$ . . . . . 36

16.5 15.5 Formula Type Dispatch Completeness . . . . . 36
16.6 15.6 Mailbox Message Number Monotonicity . . . . . 37

**17 16. Summary of Session Type Mappings 37**

**18 17. Where Prologos Stands: System Modelling and Protocol Verification 38**
18.1 17.1 What Prologos Can Do Today . . . . . 38
18.2 17.2 What Prologos Is Building Toward . . . . . 39
18.3 17.3 The Unique Value Proposition . . . . . 40
18.4 17.4 Toward an Endo Verification Story . . . . . 41

**19 18. Theoretical Connections 41**
19.1 18.1 OCap and Session Types . . . . . 41
19.2 18.2 Promise Pipelining and Async Sessions . . . . . 42
19.3 18.3 Formula Graph as Process Calculus . . . . . 42

**20 19. CRDT-Based Retention for Distributed Formula References 43**
20.1 19.1 The Partition Problem . . . . . 43
20.2 19.2 Why Reference Counting Fails Under Partition . . . . . 43
20.3 19.3 CRDT Design for Retention State . . . . . 44
20.4 19.4 Session Type for CRDT-Aware Retention . . . . . 45
20.5 19.5 Impact on GC . . . . . 46
20.6 19.6 Why This Matters for Revocation . . . . . 47

**21 20. Practical Recommendations for the Endo Codebase 48**
21.1 20.1 Immediate / Low Effort . . . . . 48
21.1.1 20.1.1 Explicit State Machine for Formula Lifecycle . . 48
21.1.2 20.1.2 Exhaustive Formula Type Dispatch . . . . . 49
21.1.3 20.1.3 Pin Lifecycle Tracking . . . . . 50
21.2 20.2 Medium Effort . . . . . 51
21.2.1 20.2.1 Operation Ordering Assertions . . . . . 51
21.2.2 20.2.2 RemoteControl State Machine Hardening . . . . 52
21.2.3 20.2.3 Structured Protocol Logging . . . . . 52
21.3 20.3 Larger Architectural Shifts . . . . . 53
21.3.1 20.3.1 CRDT Retention Layer . . . . . 53
21.3.2 20.3.2 TypeScript Branded Types for FormulaIdentifier 54
21.3.3 20.3.3 Protocol Conformance Testing . . . . . 55

**22 21. Conclusion 55**

**23 References** **56**

# 1 Abstract

Endo is a runtime framework for supply-chain attack resistance in JavaScript, built on Hardened JavaScript (SES) and the object-capability (OCap) security model. At its core is a **formula graph** — a persistent, directed graph of typed formulas representing every capability, process, agent, and communication channel managed by the Endo daemon. The formula graph coordinates capability lifecycle, distributed garbage collection, process isolation, inter-agent messaging, and capability transport across network boundaries via CapTP (Capability Transport Protocol).

This document demonstrates that Endo's interacting protocols can be expressed as composable session types in Prologos, using the protocols-as-types composition mechanism described in `PROTOCOLS_AS_TYPES.org`. We pay special attention to the asynchronous, eventual-send semantics pervasive in Endo, using Prologos's `!!` (async send) and `??` (async recv) operators to capture the non-blocking, promise-pipelined nature of capability interactions. Capabilities track authority through every protocol transition via Prologos's linear types and capability type annotations.

The analysis identifies several protocol-level properties — including mutex re-entrancy hazards, GC ordering constraints, and the implicit state machine governing formula lifecycle — that session types make explicit and statically verifiable.

# 2 1. Endo Architecture Overview

## 2.1 1.1 What Is Endo?

Endo is part of the Agoric ecosystem for hardened JavaScript. It provides:

- **Hardened JavaScript (SES)**: A locked-down JavaScript environment where all primordials are frozen, ambient authority (filesystem, network, `process.env`) is removed, and code executes in isolated `Compartment` scopes with explicitly granted endowments.
- **Object-Capability Security**: Authority is represented by object references. Holding a reference *is* having the capability. References cannot be forged, guessed, or extracted from strings. To gain a capability, it must be *granted* by someone who already holds it.

- **Persistent Daemon**: The Endo daemon manages a formula graph on disk, persisting all capabilities, agents, workers, and naming relationships across restarts. Users interact via CLI commands (`endo eval`, `endo send`, `endo invite`) that connect to the daemon over Unix domain sockets.
- **Distributed Capabilities**: CapTP enables capability references to cross process and network boundaries, maintaining the OCap discipline over the wire via marshalled slot references and promise pipelining.

```
User                    CLI                    Daemon                   Worker(s)
 |                       |                       |                        |
 |--- endo eval ------->|                       |                        |
 |                       |--- Unix socket ------>|                         |
 |                       |    CapTP handshake    |                         |
 |                       |<-- bootstrap cap -----|                         |
 |                       |                       |--- fork process ----->|
 |                       |                       |    CapTP over pipes    |
 |                       |                       |--- CTP_CALL -------->|
 |                       |                       |<-- CTP_RETURN -------|
 |                       |<-- result ------------|                         |
 |<-- output ------------|                        |                        |
```

## 2.2 1.2 Key Participants

| Role | Description | Endo Name |
|---|---|---|
| Daemon | Central process managing the formula graph, agents, workers, networking, and persistence | `EndoDaemon` |
| Host | A fully-powered agent that can create workers, guests, evaluate code, manage invitations, and grant authority | `EndoHost` |
| Guest | A reduced-authority agent that must request evaluation and capability grants through its host | `EndoGuest` |
| Worker | An isolated Compartment for code execution; communicates with the daemon via CapTP over stdin/stdout pipes | `EndoWorker` |
| Client | A CLI or programmatic client connecting via Unix socket | `EndoClient` |
| Peer | A remote Endo daemon connected via TCP/libp2p | `EndoPeer` |

## 2.3 1.3 Key Data Types

```
;; --- Endo Core Identifier Types ---
```

```
;; FormulaNumber and NodeNumber are 256-bit hex strings (64 chars)
type FormulaNumber := String    ;; identifies a formula within a node
type NodeNumber := String       ;; identifies a node (Ed25519 pubkey)

;; FormulaIdentifier = "FormulaNumber:NodeNumber"
type FormulaIdentifier := String

;; Pet names are local, per-agent capability aliases
type PetName := String

;; Special names are built-in references (SELF, HOST, AGENT, ENDO, etc.)
type SpecialName := :SELF | :HOST | :AGENT | :ENDO | :KEYPAIR
                  | :MAIN | :NETS | :PINS | :INFO | :NONE | :MAIL

;; CapTP slot references
type SlotId
  := ExportSlot Nat        ;; o+N - locally exported object
   | ImportSlot Nat         ;; o-N - imported remote object
   | ExportPromise Nat      ;; p+N - locally exported promise
   | ImportPromise Nat      ;; p-N - imported remote promise
   | Question Nat           ;; q-N - pending call awaiting answer

;; Power attenuation levels
type PowerLevel := :NONE | :SELF | :AGENT | :ENDO | :HOST
```

## 2.4 1.4 Formula Types

The formula graph contains 26+ typed formulas. Each formula is a JSON record with a `type` discriminant and formula-specific fields:

```
;; --- Formula Type Hierarchy ---
;; Formulas are the nodes of the formula graph.
;; Each formula describes HOW to construct/reconstruct a capability.

type Formula
  ;; --- Agents & Identity ---
  := EndoFormula                                ;; root of the system
   | HostFormula PetStoreId MailboxId WorkerId PrivateStoreId
   | GuestFormula HostId PetStoreId MailboxId
```

```
| DirectoryFormula PetStoreId
;; --- Code Execution ---
| EvalFormula WorkerId Source [List PetName] [List PetName] PowerLevel
| WorkerFormula                              ;; sandboxed Compartment
| ReadableBlobFormula Content                ;; stored binary data
| BundleFormula ReadableId                   ;; importable module bundle
;; --- Communication ---
| HandleFormula FormulaIdentifier            ;; local or remote reference
| PeerFormula NetworkId NodeNumber           ;; remote node connection
| LoopbackFormula                            ;; self-connection
;; --- Persistence ---
| PetStoreFormula                            ;; name -> formula mapping
| PetInspectorFormula PetStoreId             ;; read-only view of a store
| MailboxFormula PetStoreId                  ;; message storage
;; --- Promises ---
| PromiseFormula StoreId                     ;; deferred value
| ResolverFormula StoreId                    ;; resolution authority
;; --- Networking ---
| InvitationFormula NetworkId [List Byte]    ;; capability invite token
| ChannelFormula                             ;; bidirectional pipe
;; --- Authority ---
| LeastAuthorityFormula                      ;; the NONE capability
| LookupFormula HubId PetName               ;; deferred name resolution
;; --- Messages ---
| RequestFormula SenderNum RecipientNum Description [List PetName]
| PackageFormula SenderNum RecipientNum [List String] [Map String FormulaIdentifier]
```

# 3 2. The Formula Graph

The formula graph is the central data structure of the Endo daemon. It is a persistent directed graph where nodes are typed formulas and edges represent dependencies and naming relationships.

## 3.1 2.1 Graph Structure

```
EndoFormula  (permanent root)
type: endo

      depends
```

```
            v          v          v

       HostForm. PinForm NetForm
       type:host  :pins   :nets

          depends

   v     v     v         v

PetSt.WkrMailbxPvtSt.
:store:wk:mail :pvtSt.

    dynamic (pet names)
   v

  user-defined caps
  (eval, handle, ...)
```

There are two kinds of edges:

1. **Static formula dependencies**: Derived from the formula definition. A `HostFormula` depends on its `petStore`, `mailbox`, `worker`, and `privateStore`. These are immutable once the formula is created.
2. **Dynamic pet-store edges**: When a user writes a pet name (`endo store my-app`), a dynamic edge is created from the pet store formula to the target formula. These edges can be added, renamed, and removed at any time.

### 3.2 2.2 Union-Find Groups

Certain formulas share identity — they are semantically "the same thing" and must be garbage-collected atomically:

- **Handle + Agent**: A `HandleFormula` paired with its `HostFormula` or `GuestFormula` form a single capability identity.
- **Promise + Resolver**: A `PromiseFormula` and its `ResolverFormula` share a `store` identifier. The promise observation and the resolution authority are two facets of one identity.

The formula graph uses a union-find data structure with path compression and union-by-size to manage these identity groups. Garbage collection operates at the group level — if any formula in a group is reachable, the entire group is retained.

### 3.3 2.3 Persistence Invariant

A critical invariant: **disk before graph**. The formula JSON is persisted to the filesystem *before* entering the in-memory maps. This ensures that a daemon crash at any point during formula creation leaves the system in a recoverable state — the formula either exists fully on disk (and will be reconstructed on restart) or does not exist at all.

### 3.4 2.4 Single Async Mutex

All mutations to the formula graph are serialized through a single async mutex (`formulaGraphJobs`). Operations are enqueued and executed sequentially. The mutex includes re-entrancy detection — if a formula operation attempts to re-enter the mutex from within its own evaluation, this is detected and reported.

This is a crucial design point. The single mutex ensures that the graph is always in a consistent state between operations, but it also means that all formula operations are sequentialized. This is where session types can contribute: by making the sequential protocol *explicit*, we can verify that operations are correctly ordered without relying solely on runtime mutex enforcement.

## 4 3. Session Type Notation

Before diving into the protocol models, we summarize the Prologos session type operators used throughout this document:

| Operator | Meaning | Example |
|---|---|---|
| `!` | Synchronous send | `! String` |
| `?` | Synchronous receive | `? Int` |
| `!!` | Asynchronous (buffered) send | `!! EvalRequest` |
| `??` | Asynchronous (buffered) receive | `?? Result` |
| `!:` | Dependent send (binds in cont.) | `!: n Nat` |
| `?:` | Dependent receive (binds in cont.) | `?: n Nat` |
| `+>` | Internal choice (select a branch) | `+> \vert :ok -> ...` |
| `&>` | External choice (offer branches) | `&> \vert :ok -> ...` |
| `rec` | Recursive session | `rec X. ... X` |
| `end` | Session termination | |
| `[Cap C]` | Capability annotation | `[Cap FileWrite]` |

The `!!` / `??` operators are essential for modelling Endo. In Endo, nearly all inter-object communication is *eventual* — method calls go through `E()` (the eventual-send proxy), which returns a promise rather than blocking. This maps directly to `!!/??` in session types: the sender fires and continues immediately; the receiver will observe the value eventually but not necessarily synchronously.

**Naming convention**: Each named session type in this document corresponds to one identifiable protocol or sub-protocol in the Endo codebase. Session names are capitalized and use hyphens (`CapTP-Call`, `Formula-Lifecycle`).

# 5 4. CapTP: Capability Transport Protocol

CapTP is Endo's wire protocol for distributed capability communication. It implements the OCapN (Object Capability Network) standard. Every CLI-to-daemon, daemon-to-worker, and daemon-to-daemon interaction flows through CapTP.

## 5.1 4.1 Transport Layers

CapTP is transport-agnostic. It requires only a message-passing channel:

```
Application:  E(remoteObj).method(args)    (eventual send)
     |
     v
CapTP:        CTP_CALL { questionID, target, method, args }
     |
     v
```

```
Marshal:      toCapData(args) -> { body: JSON, slots: [SlotId] }
     |
     v
Framing:      Netstring encoding (length-prefixed message boundaries)
     |
     v
Transport:    Unix socket / stdin-stdout pipes / TCP / libp2p
```

### 5.2 4.2 The CapTP Message Protocol

```
;; --- CapTP Messages ---

type CapTP-Message
  := CTP-Bootstrap QuestionId
   | CTP-Call QuestionId SlotId MethodName [List CapData]
   | CTP-Return AnswerId CapTP-Result
   | CTP-Resolve PromiseSlotId CapTP-Resolution
   | CTP-Drop SlotId Nat           ;; slot + decrement count
   | CTP-Disconnect DisconnectReason
   | CTP-TrapIterate QuestionId SerializedData

type CapTP-Result
  := CallFulfilled CapData
   | CallRejected CapData

type CapTP-Resolution
  := PromiseFulfilled CapData
   | PromiseRejected CapData

;; Serialized capability data: body + slots
type CapData
  :body  String                     ;; JSON with slot markers
  :slots [List SlotId]              ;; referenced capabilities
```

### 5.3 4.3 Session Types for CapTP

The CapTP protocol has two distinct phases: a bootstrap handshake, followed by a multiplexed steady-state protocol. We model these separately.

```
;; --- Bootstrap Handshake ---
;; The initiating side requests the remote's root capability.
```

```
session CapTP-Bootstrap
  !! CTP-Bootstrap         ;; send bootstrap request (async - fire and continue)
  ?? CapTP-Result           ;; receive the bootstrap capability (eventually)
  CapTP-Steady              ;; transition to steady-state protocol
```

The bootstrap is inherently asynchronous: the client sends a bootstrap request and continues setting up its local state; the daemon processes the request and returns the bootstrap capability when ready. This is why we use `!!`/`??` rather than `!`/`?`.

```
;; --- Steady-State CapTP ---
;; After bootstrap, both sides can send any CapTP message.
;; The protocol is symmetric - either side can initiate calls.
;; Multiple concurrent calls are tracked by questionID.

session CapTP-Steady
  rec Steady
    +>                                  ;; internal choice: what to send
      | :call ->
          !! CTP-Call                   ;; send method call (async)
          ?? CapTP-Result               ;; receive result (eventually)
          Steady
      | :resolve ->
          !! CTP-Resolve                ;; notify promise settlement
          Steady
      | :drop ->
          !! CTP-Drop                   ;; GC: decrement reference count
          Steady
      | :disconnect ->
          !! CTP-Disconnect             ;; terminate connection
          end
      | :receive ->                     ;; also offering to the remote
          &>
            | :remote-call ->
                ?? CTP-Call             ;; receive method call
                !! CapTP-Result         ;; send result
                Steady
            | :remote-resolve ->
                ?? CTP-Resolve          ;; receive promise resolution
```

```
    Steady
| :remote-drop ->
    ?? CTP-Drop              ;; receive GC signal
    Steady
| :remote-disconnect ->
    ?? CTP-Disconnect        ;; receive disconnection
    end
```

**Key observation**: CapTP is a *multiplexed* protocol. Multiple concurrent calls share a single connection, each tracked by `questionID`. A more precise model would use *indexed session types* where each `questionID` has its own sub-session:

```
;; --- Indexed CapTP Call ---
;; Each questionID corresponds to an independent call sub-session.

session CapTP-Call {qid : QuestionId}
  !! [CTP-Call qid]              ;; send call with this question ID
  ?? [CapTP-Result qid]          ;; receive result for this question ID
  end                             ;; sub-session complete
```

The multiplexing structure is where Endo's *promise pipelining* lives: `E(E(x).foo()).bar()` sends *two* `CTP_CALL` messages immediately, without waiting for the first to resolve. The second call targets the *promise slot* (`p-N`) of the first call's result. This is type-safe because the session type for the pipelined call accepts a promise slot as its target.

### 5.4 4.4 Slot Duality

CapTP's slot scheme has a beautiful duality structure:

```
Side A                    Side B

o+0 (my export)         o-0 (their import)
o-1 (my import)         o+1 (their export)
p+2 (my promise)        p-2 (their promise)
q-3 (my question)       answer 3 (their answer)
```

This is exactly *session type duality*. Every export on one side is an import on the other. The `+/` prefixes are the syntactic manifestation of the dual session type. In Prologos terms, if Side A has session type `S`, then Side B has session type `dual(S)` — and the CapTP slot scheme is the runtime witness of this duality.

### 5.5 4.5 Protocol-Level Observations

**Promise pipelining and eventual-send semantics**: Every method call in Endo goes through `E()`, which is semantically an asynchronous message send. The caller gets back a *handled promise* that supports further eventual sends without waiting for the first to resolve. This is why `!!/??` is the correct model: `E(target).method(args)` is an `!! CTP-Call` that returns a promise slot, and any subsequent `E(result).next()` is another `!! CTP-Call` targeting that promise slot.

**No synchronous calls across boundaries**: CapTP has a `CTP_TRAP_ITERATE` message type for synchronous trap calls, but this is an escape hatch used only for specific debugging scenarios, not the normal protocol. The standard protocol is fully asynchronous. This aligns with the OCap principle that capabilities should not block on remote operations.

## 6 5. Formula Lifecycle Protocol

Every formula in the graph goes through a well-defined lifecycle. This lifecycle is currently implicit in the `daemon.js` code but can be made explicit as a session type.

### 6.1 5.1 The Lifecycle State Machine

```
Preformulate
 (allocate
 formula ID)

       persist to disk
      v

 Formulate
 (register
 in graph)

       create controller (sync!)
      v

 Controlled
 (controller
```

```
            exists)

                  evaluate (async)
                 v

        Active
          (value ready)


GC collect    cancel      crash

     v           v            v

          Cancelled
    (dependents notified,
     resources cleaned up)

              dispose
             v

          Disposed
    (cleanup hooks run,
     removed from graph)
```

### 6.2 5.2 Session Type for Formula Lifecycle

```
;; --- Formula Lifecycle ---
;; Models the daemon's perspective on a single formula's life.

session Formula-Lifecycle
  ;; Phase 1: Allocate a unique FormulaIdentifier
  ! FormulaIdentifier                  ;; assign formula number
  ;; Phase 2: Persist to disk (disk-before-graph invariant)
  ! Formula                            ;; write formula JSON to filesystem
  ;; Phase 3: Register in graph and create controller (SYNCHRONOUS)
  ! Controller                         ;; controller created before any await
  ;; Phase 4: Evaluate asynchronously
  !! EvalResult                        ;; async evaluation begins
  ;; Phase 5: Active - the formula is now providing its value
```

```
  Formula-Active

session Formula-Active
  rec Active
    &>                                  ;; external choice: what happens to us?
      | :provide ->
          ?? ProvideRequest             ;; someone requests our value
          !! Value                      ;; we provide it (memoized)
          Active
      | :cancel ->
          ?? CancelSignal               ;; cancellation requested
          Formula-Teardown
      | :gc-collect ->
          ?? GCCollect                  ;; garbage collector reached us
          Formula-Teardown

session Formula-Teardown
  ;; Notify dependents of cancellation
  !! CancelCascade                      ;; thisDiesIfThatDies dependents
  ;; Run cleanup hooks
  !! DisposeSignal                      ;; onCancel hooks execute
  ;; Remove from graph
  ! FormulaRemoval                      ;; delete from in-memory maps + disk
  end
```

### 6.3 5.3 Critical Ordering: Synchronous Controller Creation

A subtle but important invariant: controllers are created *synchronously* (before any `await`) during formula registration. The Endo source comments explain why:

> "The controller must be created synchronously to ensure that a re-entrant call to `provide` during the same formula graph job does not fail to find the controller."

In session type terms, this means the `! Controller` step in `Formula-Lifecycle` *must not* be an `!! Controller` (async). The synchronous send ensures that the controller is visible to any concurrent `provide` call before the async evaluation begins. Session types make this ordering constraint explicit — changing `!` to `!!` would be a type error if any downstream protocol depends on the controller's synchronous availability.

### 6.4 5.4 Transient Pinning

Newly created formulas face a race condition: between creation and the establishment of a pet-name reference, the formula has no incoming edges and could be garbage-collected. Endo resolves this with *transient pinning*:

```
;; --- Pin Protocol ---
;; Protects a formula from GC during the window between
;; creation and naming.

session Pin-Guard
  ! FormulaIdentifier            ;; pin the formula
  Pin-Protected

session Pin-Protected
  rec Pinned
    +>
      | :name-established ->
          ! PetName              ;; the formula now has a named reference
          ! Unpin                ;; safe to unpin
          end
      | :abandon ->
          ! Unpin                ;; no name was established; formula may be GC'd
          end
```

**Protocol-level issue**: The pin protocol has no *timeout*. If a formula is pinned and the naming step never completes (e.g., due to a crash between `pinTransient` and the pet-store write), the pin leaks. Session types could enforce a bounded liveness property here — requiring that every `Pin-Guard` eventually reaches `Unpin`.

## 7 6. Agent Protocols

Endo has two primary agent types, `Host` and `Guest`, with the Guest being a capability-attenuated version of the Host.

### 7.1 6.1 Host Agent

The Host is the most powerful agent type. It can create workers, evaluate code, manage guests, handle invitations, and access the full Endo system.

```
;; --- Host Agent Protocol ---
;; The host is the "root of authority" for its subtree of capabilities.

session Host-Protocol
  rec HostOps
    &>                                    ;; offer to handle requests
      ;; --- Code Execution ---
      | :evaluate ->
          ?? EvalRequest                  ;; receive: worker, source, endowments
          !! EvalResult                   ;; async: result of evaluation
          HostOps
      | :make-unconfined ->
          ?? UnconfinedRequest            ;; receive: specifier, powers
          !! EvalResult                   ;; async: imported module result
          HostOps
      | :make-bundle ->
          ?? BundleRequest                ;; receive: readable, powers
          !! EvalResult                   ;; async: bundle evaluation result
          HostOps
      ;; --- Agent Management ---
      | :make-host ->
          ?? HostRequest                  ;; create a new sub-host
          !! FormulaIdentifier            ;; async: the new host's ID
          HostOps
      | :make-guest ->
          ?? GuestRequest                 ;; create a reduced-authority guest
          !! FormulaIdentifier            ;; async: the new guest's ID
          HostOps
      ;; --- Capability Management ---
      | :store-value ->
          ?? StoreValueRequest            ;; receive: pet name + value
          ! Acknowledgment               ;; stored (sync - disk write)
          HostOps
      | :store-blob ->
          ?? StoreBlobRequest             ;; receive: pet name + bytes
          ! Acknowledgment
          HostOps
      | :lookup ->
          ?? LookupRequest               ;; receive: pet name to resolve
          !! Value                        ;; async: the resolved capability
```

```
    HostOps
| :remove ->
    ?? RemoveRequest                ;; receive: pet name to delete
    ! Acknowledgment                ;; GC may trigger
    HostOps
;; --- Invitation Protocol ---
| :invite ->
    ?? InviteRequest                ;; create an invitation
    !! Invitation                    ;; async: invitation bytes
    HostOps
| :accept ->
    ?? AcceptRequest                ;; accept an invitation
    !! FormulaIdentifier            ;; async: the new peer's handle
    HostOps
;; --- Messaging ---
| :send ->
    ?? SendRequest                  ;; send a message/package
    ! Acknowledgment
    HostOps
| :request ->
    ?? RequestMsg                   ;; send a request (expects response)
    ?? Resolution                   ;; await resolution
    HostOps
;; --- Lifecycle ---
| :cancel ->
    ?? CancelRequest                ;; cancel a named capability
    ! Acknowledgment
    HostOps
```

### 7.2 6.2 Guest Agent

The Guest is a capability-attenuated view. It *cannot* directly create workers, hosts, or guests. Instead, it must *request* evaluation through a messaging protocol that the host (or an evaluator agent) must approve.

```
;; --- Guest Agent Protocol ---
;; Guests have a reduced method set compared to hosts.
;; They can evaluate code only if the host grants permission.

session Guest-Protocol
```

```
  rec GuestOps
    &>
      | :request-eval ->
          ?? EvalRequestFromGuest        ;; guest wants to evaluate code
          !! RequestAcknowledgment       ;; the request is registered
          ;; The host will receive a message and must approve.
          ;; The guest awaits the result via its mailbox.
          GuestOps
      | :lookup ->
          ?? LookupRequest
          !! Value
          GuestOps
      | :store-value ->
          ?? StoreValueRequest
          ! Acknowledgment
          GuestOps
      | :list ->
          ?? ListRequest
          !! [List PetName]
          GuestOps
      | :follow-names ->
          ?? FollowRequest
          Guest-NameStream
      | :send ->
          ?? SendRequest
          ! Acknowledgment
          GuestOps
      | :request ->
          ?? RequestMsg
          ?? Resolution
          GuestOps

session Guest-NameStream
  rec Stream
    ?? NameChange                        ;; receive name change notification
    Stream
```

### 7.3 6.3 Capability Attenuation: Host → Guest

The Host-to-Guest relationship is a classic OCap *attenuation* pattern. The Guest has strictly fewer methods than the Host. In Prologos's session type system, this is captured by *subtyping*: `Guest-Protocol` is a subtype of `Host-Protocol` (it offers a subset of the choices). Any code that works with a `Guest-Protocol` session will also work with a `Host-Protocol` session, but not vice versa.

```
;; The subtyping relationship (conceptual):
;; Host-Protocol <: Guest-Protocol   (Host offers MORE choices)
;;
;; But session subtyping inverts for offers (&>):
;; If Guest offers fewer options, Host is "wider" - so
;; Host-Protocol is a subtype of Guest-Protocol for clients.
;;
;; This captures the OCap attenuation: a Host can be used
;; anywhere a Guest is expected, but not the reverse.
```

## 8 7. Worker Isolation Protocol

Workers are sandboxed JavaScript execution environments. They run in separate OS processes, communicating with the daemon exclusively through CapTP over stdin/stdout pipes.

```
;; --- Worker Protocol ---
;; The daemon is the sole initiator; the worker is purely reactive.

session Worker-Protocol
  rec WorkerOps
    +>                                  ;; daemon chooses what to send
      | :evaluate ->
          !! EvalCommand                ;; send: source, names, values
          ?? EvalResult                 ;; receive: evaluation result
          WorkerOps
      | :make-unconfined ->
          !! UnconfinedCommand          ;; send: specifier, powers
          ?? EvalResult
          WorkerOps
      | :make-bundle ->
```

```
        !! BundleCommand                ;; send: readable, powers
        ?? EvalResult
        WorkerOps
    | :terminate ->
        !! TerminateCommand             ;; send: shutdown signal
        end
```

The worker protocol has an important asymmetry: *the worker never initiates a call.* The daemon holds the worker facet (received as the worker's CapTP bootstrap), and all interaction is daemon-to-worker. The worker can only respond to calls.

This asymmetry is captured by the session type: the daemon side uses `+>` (internal choice — it decides which operation to perform), while the dual worker side uses `&>` (external choice — it must be prepared to handle any operation the daemon selects).

```
;; --- Worker Sandboxing ---
;; The Compartment endowments define the worker's capability surface.

type WorkerEndowments
  :assert     AssertCapability
  :console    ConsoleCapability
  :e          EventualSendProxy
  :far        FarMaker
  :make-exo   ExoMaker
  :m          MethodGuard
  :text-encoder TextEncoderCap
  :text-decoder TextDecoderCap
  :url        URLConstructor

;; Note: The worker does NOT receive filesystem, network, or process
;; capabilities. All authority must come through the capability arguments
;; passed in the evaluate/makeUnconfined/makeBundle calls.
```

# 9 8. Pet Store: Capability Naming Protocol

The pet store is Endo's capability naming layer. Each agent has its own pet store with its own names — there is no global namespace. This implements the *pet name* pattern from capability security literature.

## 9.1 8.1 Core Protocol

```
;; --- Pet Store Protocol ---
;; A local, per-agent namespace for capabilities.

session PetStore-Protocol
  rec StoreOps
    &>
      | :write ->
          ?? WriteRequest                  ;; { petName, formulaIdentifier }
          ! WriteAck                       ;; persisted to disk
          StoreOps
      | :remove ->
          ?? RemoveRequest                 ;; { petName }
          ! RemoveAck                      ;; edge removed, GC may trigger
          StoreOps
      | :rename ->
          ?? RenameRequest                 ;; { fromName, toName }
          ! RenameAck
          StoreOps
      | :identify ->
          ?? IdentifyRequest               ;; { petName }
          ! [Option FormulaIdentifier]     ;; Some if found, None if not
          StoreOps
      | :reverse-identify ->
          ?? ReverseIdRequest              ;; { formulaIdentifier }
          ! [List PetName]                 ;; all names for this formula
          StoreOps
      | :list ->
          ?? ListRequest
          ! [List PetName]                 ;; sorted list of all names
          StoreOps
      | :follow ->
          ?? FollowRequest
          PetStore-ChangeStream
      | :follow-id ->
          ?? FollowIdRequest               ;; { formulaIdentifier }
          PetStore-IdChangeStream

session PetStore-ChangeStream
```

```
  rec Changes
    !! NameChange                          ;; async: push change notification
    Changes

type NameChange
  := NameAdded PetName FormulaIdentifier
   | NameRemoved PetName FormulaIdentifier
```

### 9.2 8.2 Bidirectional Mapping

The pet store maintains a bidirectional mapping: name → formulaId *and* formulaId → [names]. This enables both `identifyLocal` (name → id) and `reverseIdentify` (id → names) in O(1). The session type exposes both directions as separate protocol steps.

### 9.3 8.3 Protocol-Level Observation: Write Ordering

Pet store writes are persisted to the filesystem. The `WriteAck` is sent *after* the disk write completes. This is a synchronous `!` (not `!!`) because the caller needs to know that the name is durably recorded before proceeding. If we changed this to `!! WriteAck` (async), we would lose the persistence guarantee — a crash between the async ack and the actual write would leave the in-memory mapping inconsistent with disk.

This is a good example of where the choice between `!` and `!!` in the session type encodes a *correctness requirement*, not just a performance preference.

## 10 9. Mail and Messaging Protocol

The mail system enables agents to communicate with each other. Messages carry capability references (as formula identifiers), enabling the *capability introduction* pattern: Agent A introduces a capability to Agent B by including it as a message edge.

### 10.1 9.1 Message Types

```
;; --- Message Type Hierarchy ---

type Message
  := RequestMessage Nat Nat Description [List PetName]
```

```
  | PackageMessage Nat Nat [List String] [Map String FormulaIdentifier]
  | EvalRequestMessage Nat Nat Source [List PetName] [List PetName] PetName
  | DefinitionMessage Nat FormulaIdentifier PetName
  | FormMessage Nat Nat Description [List String]
  | ValueMessage Nat Nat Value
  | EvalProposalReviewer Nat Nat Source [List PetName] [List PetName] PetName
  | EvalProposalProposer Nat Nat Source [List PetName] [List PetName] PetName
```

## 10.2 9.2 Mail Protocol

```
;; --- Mailbox Protocol ---
;; Agents send and receive messages through their mailbox.

session Mailbox-Send
  +>
    | :send-package ->
        !! PackageMessage                  ;; async: send a package to a recipient
        end
    | :send-request ->
        !! RequestMessage                  ;; async: send a request
        ?? Resolution                      ;; await: resolution from recipient
        end
    | :send-eval-request ->
        !! EvalRequestMessage              ;; async: request code evaluation
        ?? EvalResult                      ;; await: evaluation result
        end
    | :send-value ->
        !! ValueMessage                    ;; async: send a raw value
        end
    | :send-form ->
        !! FormMessage                     ;; async: send an interactive form
        ?? FormSubmission                  ;; await: submitted values
        end

session Mailbox-Receive
  rec Inbox
    ?? Message                             ;; receive any message type
    +>                                     ;; choose how to handle it
      | :adopt ->
          ! AdoptRequest                   ;; claim a capability edge
```

```
        Inbox
    | :resolve ->
        ! ResolveResponse                ;; fulfill a request
        Inbox
    | :reject ->
        ! RejectResponse                 ;; reject a request
        Inbox
    | :dismiss ->
        ! DismissSignal                  ;; discard the message
        Inbox
    | :submit ->
        ! FormSubmission                 ;; submit form values
        Inbox
    | :grant-evaluate ->
        ! GrantSignal                    ;; approve an eval proposal
        Inbox
    | :counter-evaluate ->
        ! CounterProposal               ;; send a counter-proposal
        Inbox
```

### 10.3 9.3 Capability Introduction Pattern

When Agent A sends a message to Agent B that includes capability references:

```
;; --- Capability Introduction ---
;; A three-party protocol: sender, daemon, recipient.

session Capability-Introduction
  ;; Phase 1: Sender references capabilities by local pet name
  ! [List PetName]                        ;; sender's pet names

  ;; Phase 2: Daemon resolves names to formula identifiers
  ;; (internal - not visible as a protocol step to sender/recipient)

  ;; Phase 3: Message is stored with formula identifiers as edges
  ! [Map String FormulaIdentifier]        ;; edges: edgeName -> formulaId

  ;; Phase 4: Recipient adopts edges under their own pet names
  rec Adopt
```

```
  +>
    | :adopt-edge ->
        ? String                        ;; edge name in the message
        ? PetName                       ;; recipient's chosen pet name
        Adopt
    | :done ->
        end
```

This is the mechanism by which capabilities flow between agents while maintaining the pet-name discipline: each agent only sees their own names, never the other's. The daemon mediates the translation through formula identifiers.

# 11 10. Garbage Collection Protocol

The formula graph's garbage collector is a critical correctness component. It determines which formulas are still reachable and cancels/removes those that are not.

## 11.1 10.1 GC Algorithm as Protocol Steps

```
;; --- Garbage Collection Protocol ---
;; The 10-step GC sweep, modeled as a session.

session GC-Sweep
  ;; Step 1: Assign union-find groups
  ! GroupAssignment                     ;; merge handle+agent, promise+resolver

  ;; Step 2: Build dependency graph at group level
  ! [Map GroupId [Set GroupId]]         ;; group -> dependent groups

  ;; Step 3: Count references to each group
  ! [Map GroupId Nat]                   ;; group -> reference count

  ;; Step 4: Identify root set + reachable groups
  ! [Set GroupId]                       ;; permanently rooted groups
  ! [Set GroupId]                       ;; transitively reachable groups

  ;; Step 5: Identify unreachable groups for collection
  ! [Set GroupId]                       ;; groups to collect
```

```
  ;; Step 6: Cancel unreachable formulas (cascading)
  !! CancelBatch                          ;; async: cancel signals sent

  ;; Step 7: Drop CapTP references for collected formulas
  !! DropBatch                            ;; async: CTP_DROP sent to peers

  ;; Step 8: Disconnect CapTP sessions for collected formulas
  !! DisconnectBatch                      ;; async: CTP_DISCONNECT sent

  ;; Step 9: Delete formulas from disk
  ! DeleteBatch                           ;; sync: filesystem deletion

  ;; Step 10: Clear dirty flag
  ! GCComplete
  end
```

### 11.2 10.2 GC Trigger Protocol

GC does not run continuously. It is triggered by `withCollection`, a wrapper that runs a user-facing operation and then triggers collection if the graph is dirty:

```
;; --- withCollection ---
;; Every capability-affecting operation is wrapped in withCollection.

session WithCollection {A : Type}
  ! Operation                             ;; the user-facing operation
  ?? A                                    ;; its result (async)
  +>
    | :graph-dirty ->
        GC-Sweep                          ;; trigger collection
    | :graph-clean ->
        end                               ;; no collection needed
```

### 11.3 10.3 Protocol-Level Observations

**Promise resolution ordering**: When a promise resolves, the daemon writes the resolved formula identifier as a pet store entry *before* writing the resolution status. This ensures that the GC sees the new reference edge before it sees that the promise is resolved. If this ordering were reversed,

a GC sweep between the status write and the edge write could collect the target formula. Session types make this ordering constraint explicit:

```
;; --- Promise Resolution Ordering ---
;; The edge must be written before the status, or GC could
;; collect the target formula.

session Promise-Resolution-Safe
  ! PetStoreEdge                          ;; write: name -> resolved formula (FIRST)
  ! ResolutionStatus                      ;; write: promise is resolved (SECOND)
  end

;; INCORRECT ordering would be:
;; session Promise-Resolution-Unsafe
;;   ! ResolutionStatus                   ;; write status first - DANGER
;;   ! PetStoreEdge                       ;; GC could run between these two steps!
;;   end
```

**Distributed GC via $\mathbf{CTP_{DROP}}$**: When a formula is collected, any CapTP connections that were importing it must be notified. The `CTP_DROP` message decrements the remote reference count. If the count reaches zero, the remote side can collect its proxy. This is modeled by the `DropBatch` step in `GC-Sweep`.

## 12 11. Invitation and Peering Protocol

Endo supports cross-daemon capability sharing through an invitation protocol.

```
;; --- Invitation Protocol ---
;; A two-phase protocol for establishing a peering relationship.

;; Phase 1: The inviting host creates an invitation
session Create-Invitation
  ! NetworkId                             ;; which network to use
  ! InvitationBytes                       ;; cryptographic invitation token
  !! InvitationFormula                    ;; async: formula registered in graph
  end

;; Phase 2: The accepting host uses the invitation
```

```
session Accept-Invitation
  ? InvitationBytes                        ;; receive the invitation token
  ;; Validate the invitation against the network
  ?? PeerHandle                            ;; async: the new peer's handle
  ;; The peer is now a named formula in the accepting host's store
  ! PetName                                ;; assign a local name to the peer
  end

;; The full peering handshake composes both phases:
session Peering-Handshake
  ;; Inviting side:
  Create-Invitation
  ;; ... invitation bytes are transferred out-of-band ...
  ;; Accepting side:
  Accept-Invitation
  ;; Both sides now have CapTP connections to each other.
  CapTP-Bootstrap
```

The invitation protocol is interesting because the invitation bytes travel *out-of-band* (e.g., via email, QR code, or another communication channel). The session type cannot model the out-of-band transfer directly, but it can model the two sides independently and verify that their composed types are compatible.

## 13 12. Cancellation and Context Protocol

Each formula has a `Context` that manages its lifecycle dependencies and cancellation cascade.

```
;; --- Cancellation Context ---
;; The cascading cancellation protocol.

type ContextDependency
  := ThisDiesIfThatDies FormulaIdentifier
   | ThatDiesIfThisDies FormulaIdentifier

session Context-Protocol
  rec ContextOps
    &>
      | :add-dependency ->
```

```
          ?? ContextDependency             ;; register a liveness dependency
          ContextOps
      | :add-cancel-hook ->
          ?? CancelHook                    ;; register a cleanup callback
          ContextOps
      | :cancel ->
          ?? CancelSignal                  ;; cancellation initiated
          Context-Cancel-Cascade
      | :dispose ->
          ?? DisposeSignal                 ;; external disposal request
          Context-Dispose

session Context-Cancel-Cascade
  ;; 1. Mark as cancelled (reject the cancelled promise)
  ! CancelledNotification
  ;; 2. Cancel all dependents (thatDiesIfThisDies)
  !! [List CancelSignal]                   ;; async: cascade to dependents
  ;; 3. Run cleanup hooks
  !! [List HookResult]                     ;; async: execute onCancel hooks
  ;; 4. Resolve the disposed promise
  ! DisposedNotification
  end

session Context-Dispose
  ;; Run cleanup hooks
  !! [List HookResult]
  ! DisposedNotification
  end
```

**Protocol-level observation**: The cancellation cascade creates a directed acyclic graph of liveness dependencies. If this graph has a cycle, a formula's cancellation would cascade back to itself, potentially causing infinite recursion. Session types with recursive structure can detect such cycles statically. The `ContextDependency` type encodes the direction of the dependency — `ThisDiesIfThatDies` vs `ThatDiesIfThisDies` — and a static analysis can verify that the dependency graph is acyclic.

## 14 13. Connection and Remote Control

The `RemoteControl` type in Endo represents the state machine for a peering connection.

```
;; --- Remote Control State Machine ---
;; Tracks the lifecycle of a connection to a remote peer.

type RemoteControlState
  := Pending                              ;; connection not yet established
   | Connected CapTPConnection            ;; active connection
   | Disconnected DisconnectReason        ;; connection closed
   | Failed Error                         ;; connection failed

session RemoteControl-Protocol
  ;; Initial state: pending
  ?? ConnectionAttempt                    ;; receive connection parameters
  +>
    | :connect-success ->
        ! Connected                       ;; report connected state
        RemoteControl-Active
    | :connect-failure ->
        ! Failed                          ;; report failure
        end

session RemoteControl-Active
  rec Active
    &>
      | :send-message ->
          ?? CapTP-Message                ;; receive message to send
          !! DeliveryResult               ;; async: delivery outcome
          Active
      | :receive-message ->
          ?? CapTP-Message                ;; receive message from remote
          Active
      | :disconnect ->
          ?? DisconnectRequest
          ! Disconnected
          end
      | :connection-lost ->
```

```
        ?? ConnectionError
        ! Disconnected
        end
```

# 15 14. Composed Protocol: Full Daemon Bootstrap

The full Endo daemon bootstrap sequence composes many of the protocols above:

```
;; --- Full Daemon Bootstrap ---
;; This is the composition of all protocols involved in starting
;; the Endo daemon and processing its first client request.

session Daemon-Bootstrap
  ;; Phase 1: Formula graph initialization
  Formula-Lifecycle                        ;; create the Endo root formula
  Formula-Lifecycle                        ;; create networks formula
  Formula-Lifecycle                        ;; create pins formula
  Formula-Lifecycle                        ;; create main worker
  Formula-Lifecycle                        ;; create host agent

  ;; Phase 2: Socket listener
  ?? ClientConnection                      ;; accept Unix socket connection

  ;; Phase 3: CapTP handshake
  CapTP-Bootstrap                          ;; exchange bootstrap capabilities

  ;; Phase 4: Steady-state operation
  Daemon-Steady

session Daemon-Steady
  rec Steady
    &>
      ;; Accept new client connections
      | :new-client ->
          ?? ClientConnection
          CapTP-Bootstrap
          Steady
```

```
;; Process existing client requests
| :client-request ->
    ?? CapTP-Message
    ;; Route to appropriate agent protocol
    +>
      | :host-op -> Host-Protocol
      | :guest-op -> Guest-Protocol
      | :worker-op -> Worker-Protocol
    Steady
;; Manage lifecycle
| :gc-trigger ->
    WithCollection
    Steady
;; Shutdown
| :shutdown ->
    !! ShutdownSignal
    ;; Cancel all formulas
    GC-Sweep
    end
```

This composition demonstrates the power of named session types as protocol building blocks. Each named session (`Formula-Lifecycle`, `CapTP-Bootstrap`, `Host-Protocol`, etc.) is independently verifiable, and their composition is verified at the type level.

# 16 15. Protocol-Level Issues and Recommendations

The session type analysis reveals several protocol-level properties that are currently implicit in the Endo codebase but could benefit from being made explicit and statically verified.

## 16.1 15.1 Mutex Re-Entrancy

The formula graph mutex (`formulaGraphJobs`) serializes all graph mutations. The code includes re-entrancy detection, logging a warning if a formula operation re-enters the mutex. But this is a runtime check.

**Session type approach**: Model the mutex as a session with a *linear* acquire-release protocol:

```
;; --- Mutex Protocol ---
```

```
;; Linear typing ensures acquire/release pairing.

session Mutex-Protocol
  ? MutexToken [1]                          ;; acquire: linear - must be used exactly o
  ;; ... critical section ...
  ! MutexToken [1]                          ;; release: returns the token
  end

;; Re-entrancy is a type error: if you hold a MutexToken,
;; attempting to acquire another would require TWO tokens,
;; but the mutex session only provides one.
```

Linearity prevents re-entrancy *statically* — you cannot acquire a linear token while already holding one.

## 16.2 15.2 Pin Leak Prevention

As noted in Section 5.4, the transient pin protocol has no timeout or bounded liveness guarantee. A crash between `pinTransient` and the corresponding `unpinTransient` leaves a permanent pin.

**Session type approach**: Model the pin as a linear resource:

```
;; --- Linear Pin ---
;; The pin MUST be consumed (unpinned). Linearity prevents leaks.

session Pin-Linear
  ? PinToken [1]                            ;; acquire pin: linear
  ;; ... naming operations ...
  ! PinToken [1]                            ;; release pin: MUST happen
  end

;; Dropping a linear PinToken without consuming it is a type error.
;; This catches pin leaks at compile time.
```

## 16.3 15.3 Promise Resolution GC Ordering

Section 10.3 described the critical ordering requirement for promise resolution: the pet store edge must be written *before* the resolution status. This is currently enforced by code ordering (sequential statements), which is fragile under refactoring.

**Session type approach**: The ordering is explicit in the `Promise-Resolution-Safe` session type. Any refactoring that swaps the two writes would be a type error, because the session's continuation structure enforces the order.

### 16.4 15.4 $\text{CTP}_{\text{DROP}}$ Before $\text{CTP}_{\text{DISCONNECT}}$

During GC, the daemon must send `CTP_DROP` messages to decrement reference counts *before* sending `CTP_DISCONNECT` to close connections. If the order is reversed, the remote side may not process the drops before the connection closes, leaving stale references.

```
;; --- GC Network Cleanup Ordering ---

session GC-Network-Cleanup
  ;; Drops MUST precede disconnect
  !! [List CTP-Drop]                        ;; send all reference decrements
  !! CTP-Disconnect                         ;; THEN close the connection
  end

;; The session type enforces: Drop < Disconnect
;; Swapping them would change the session type and be caught by the checker.
```

### 16.5 15.5 Formula Type Dispatch Completeness

The daemon dispatches on formula type in `formulate` and several other functions. If a new formula type is added but a dispatch case is missed, it manifests as a runtime `undefined` error.

**Session type approach**: Model the formula dispatch as an external choice (`&>`) with one branch per formula type. The session type checker verifies that all branches are handled. Adding a new formula type without adding the corresponding handler is a type error:

```
;; --- Formula Type Dispatch ---
;; Exhaustive matching enforced by session type.

session Formula-Dispatch
  &>
    | :endo -> ...
    | :host -> ...
    | :guest -> ...
    | :eval -> ...
```

```
    | :worker -> ...
    | :handle -> ...
    | :promise -> ...
    | :resolver -> ...
    ;; ... all 26+ formula types ...
    ;; Missing a case is a type error.
```

### 16.6 15.6 Mailbox Message Number Monotonicity

Mailbox messages are numbered with a monotonically increasing `bigint` counter. But the mailbox protocol does not enforce that responses reference valid (existing) message numbers. A `resolve(999, ...)` where message 999 does not exist is a runtime error.

**Session type approach**: Use dependent session types to bind the message number at receive time and constrain it at response time:

```
;; --- Dependent Message Numbers ---

session Mailbox-Receive-Dep
  rec Inbox
    ?: msgNum Nat                          ;; dependent recv: binds msgNum
    ?? [Message msgNum]                    ;; the message, indexed by its number
    +>
      | :resolve ->
          ! [Resolution msgNum]            ;; MUST reference the same msgNum
          Inbox
      | :dismiss ->
          ! [Dismiss msgNum]
          Inbox
```

## 17 16. Summary of Session Type Mappings

| Endo Component | Session Type | Key Operators |
|---|---|---|
| CapTP handshake | `CapTP-Bootstrap` | `!! ??` |
| CapTP steady-state | `CapTP-Steady` | `!! ?? +> &>` |
| Indexed CapTP call | `CapTP-Call {qid}` | `!! ??` (dep.) |
| Formula lifecycle | `Formula-Lifecycle` | `! !! &>` |
| Transient pinning | `Pin-Guard` | `! +` |
| Host agent | `Host-Protocol` | `?? !! &>` |
| Guest agent | `Guest-Protocol` | `?? !! &>` |
| Worker sandboxing | `Worker-Protocol` | `!! ?? +>` |
| Pet store | `PetStore-Protocol` | `?? ! &>` |
| Pet store change stream | `PetStore-ChangeStream` | `!!` |
| Mailbox send | `Mailbox-Send` | `!! ?? +>` |
| Mailbox receive | `Mailbox-Receive` | `?? ! +>` |
| Capability introduction | `Capability-Introduction` | `! ? +>` |
| GC sweep | `GC-Sweep` | `! !!` |
| GC trigger | `WithCollection` | `! ?? +>` |
| Promise resolution ordering | `Promise-Resolution-Safe` | `!` |
| Cancellation cascade | `Context-Cancel-Cascade` | `! !!` |
| Invitation | `Create-/Accept-Invitation` | `! ?? !!` |
| Remote control | `RemoteControl-Protocol` | `?? !! +> &>` |
| Mutex protocol | `Mutex-Protocol` | `? ! [1]` |
| Daemon bootstrap | `Daemon-Bootstrap` | composed |
| Daemon steady-state | `Daemon-Steady` | `&> +>` |

# 18 17. Where Prologos Stands: System Modelling and Protocol Verification

## 18.1 17.1 What Prologos Can Do Today

Prologos is a Phase 0 research language with a working implementation in Racket. Its current capabilities relevant to system modelling:

- **Session Types with Full Operator Set**: Prologos implements synchronous (`!`/?=), asynchronous (`!!/??`), dependent (`!:/??:`), choice (`+>/&>`), and recursive (`rec/end`) session type operators. Every session type in this document uses syntax that Prologos already parses, type-checks, and verifies.
- **Session Type Duality**: The type checker verifies that dual endpoints have compatible session types. This directly maps to CapTP's slot duality (`o+N` / `o-N`).

- **Linear Types (QTT)**: Prologos uses Quantitative Type Theory with multiplicities (`0`, `1`, `\omega`). Linear resources (multiplicity `1`) are used exactly once. This can model the mutex token, pin token, and capability transfer patterns described in this document.
- **Dependent Types**: Prologos has full dependent types with type-level computation, enabling indexed session types (`CapTP-Call {qid}`) and dependent message binding (`?: msgNum Nat`).
- **Propagator Network Substrate**: The underlying computation model is propagator networks, supporting monotone constraint propagation, partial information, and hypothetical reasoning (ATMS). This is the foundation for the session-derived effect ordering system.
- **Capability Type Annotations**: `[Cap C]` annotations on session types track which capabilities are required for each protocol step, composing alongside the protocol itself.
- **Protocol Composition via Named Sessions**: Named session types compose through type-level continuations — exactly the mechanism used throughout this document to build complex protocols from simple building blocks.

## 18.2 17.2 What Prologos Is Building Toward

Several capabilities are in active development or planned:

- **Session-Derived Effect Ordering (Architecture A+D)**: The effect position lattice (recently completed) enables IO effects to be ordered by session type structure via a Galois connection. This means the ordering of capability operations in an Endo-like system would be derived from the session types — not from ad-hoc code ordering — and verified by the type checker.
- **Async Session Semantics**: The `!!/??` operators currently exist in the type system but their runtime semantics (buffered channels, promise pipelining) are not yet fully implemented. Full async session support would enable direct execution of the protocols modelled in this document.
- **Process Calculus Runtime**: Prologos's `proc` runtime supports multi-channel processes, but does not yet have network-transparent capability passing. Adding CapTP-like capability transport at the runtime

level would enable Prologos processes to interact with Endo-style systems.

- **Static Deadlock Detection**: The transitive closure computation in the effect position lattice can detect cycles in cross-channel data dependencies, turning runtime deadlocks into compile-time errors. Applied to an Endo-like formula graph, this would detect circular liveness dependencies in the cancellation context.
- **Refinement Types for Protocol Properties**: With dependent types, Prologos can express refinement predicates over session states — e.g., "this message number refers to an existing message" or "this slot ID refers to an exported object." These go beyond standard session types into property-rich protocol specifications.

## 18.3 17.3 The Unique Value Proposition

What makes Prologos uniquely suited to modelling systems like Endo is the *convergence* of multiple type system features:

1. **Session types** capture the communication protocol structure.
2. **Linear types** capture the resource discipline (capability transfer, mutex acquire/release, one-shot tokens).
3. **Dependent types** capture data-dependent protocol branching (indexed sessions, message-number-scoped responses).
4. **Capability annotations** track authority through protocol transitions.
5. **Propagator networks** provide the compositional, monotone reasoning substrate for effect ordering and constraint solving.

No other system currently combines all five. Existing tools offer subsets:

- **Scribble/mpst**: Session types, but no dependent types, linear types, or capability tracking.
- **Linear Haskell**: Linear types, but session types require encoding tricks and dependent types are limited.
- **Idris 2**: Dependent types with QTT multiplicities, but session types require encoding as indexed monads.

- **Rust**: Ownership/borrowing (affine types), but no session types or dependent types in the type system.

Prologos aims to be the first language where all five features are native and compose naturally through the type system.

### 18.4 17.4 Toward an Endo Verification Story

A fully realized Prologos could provide Endo with:

- **Protocol specification language**: Define each CapTP message flow, formula lifecycle, and agent interaction as a session type. The specification serves as both documentation and executable verification.
- **Exhaustive dispatch verification**: Session type choices (`&>`) ensure that every formula type, message type, and protocol branch is handled. Missing a case is a compile-time error.
- **Linear resource safety**: Capabilities that must be used exactly once (invitations, pins, mutex tokens) are tracked by the type system. Dropping a linear capability is a type error. Using it twice is a type error.
- **Ordering constraint verification**: The session-derived effect ordering ensures that operations like "write edge before status" are verified by the type checker, not by code review.
- **Cross-agent protocol compatibility**: When Agent A sends a message to Agent B, their session types must be *dual*. The type checker verifies this, catching protocol mismatches before runtime.
- **Composition verification**: When protocols are composed (`Daemon-Bootstrap` chains `Formula-Lifecycle`, `CapTP-Bootstrap`, `Host-Protocol`), the composed session type is well-formed if and only if all the named sub-sessions are compatible at their boundaries.

## 19 18. Theoretical Connections

### 19.1 18.1 OCap and Session Types

Object-capability security and session types share a deep connection through linear logic. Both enforce:

- **Authority is held, not named**: In OCap, holding a reference is having the capability. In session types, holding a channel endpoint is having the communication capability.
- **Transfer is explicit**: In OCap, capabilities are granted by passing references. In session types, channel delegation (`!` / `?` of a channel) explicitly transfers communication authority.
- **Attenuation via typing**: In OCap, a restricted facet attenuates authority. In session types, a subtype (offering fewer choices) attenuates the protocol.

Caires and Pfenning's foundational work connecting session types to linear logic propositions (Curry-Howard for sessions) provides the theoretical bridge: a session type *is* a linear logic proposition, and linear logic is the logic of resources — the same resource discipline that underlies OCap.

## 19.2 18.2 Promise Pipelining and Async Sessions

Endo's promise pipelining (`E(E(x).foo()).bar()` sends both calls immediately) is precisely the operational semantics of async session types. In a session with `!! A . !! B`, both sends can be issued without waiting for acknowledgment. The session type guarantees that they will be received in order on the dual side, but the sender does not block.

This connection is not accidental. Mark Miller's E language — the direct ancestor of Endo's `E()` eventual-send proxy — was designed with exactly this pipelining semantics. Async session types formalize what E provided operationally.

## 19.3 18.3 Formula Graph as Process Calculus

The Endo formula graph can be viewed as a *process calculus* configuration: each formula is a process, each edge is a channel, and formula lifecycle operations (create, evaluate, cancel) are process creation, execution, and termination. The formula graph's GC is process-level garbage collection — collecting processes that are no longer reachable through any channel.

This process calculus view is exactly what Prologos's `proc` runtime provides. The connection suggests a natural embedding: Endo formulas as Prologos processes, formula identifiers as channel names, and CapTP messages as session-typed communications.

# 20 19. CRDT-Based Retention for Distributed Formula References

A hypothesis contributed by an Endo maintainer: *the formula system is missing a CRDT for managing retention of local formulas on a pair of connected peers that must be able to progress when partitioned, notably for deletion of references, as this is the basis for revocation.*

This section analyzes this hypothesis through the lens of our session type models and finds it well-founded.

## 20.1 19.1 The Partition Problem

The formula graph currently assumes a centralized, connected model. Reference counting between peers flows through `CTP_DROP` messages, and GC is a local sweep that runs under the single async mutex. When two peers are connected, this works:

```
Peer A                    Peer B

Host holds handle →       Guest has o-N import
Host cancels formula →    Sends CTP_DROP → Guest releases
GC sweep →                Reference count = 0 → Collect
```

Under partition, this breaks:

```
Peer A                      Peer B

Host cancels formula        Guest still holds o-N import
Wants to send CTP_DROP      Network down - message never arrives
GC sweep sees no remote     Guest may pass capability to Carol
  refs (can't reach B)        via promise pipelining

Dilemma:
- GC eagerly?               → Dangling reference when B reconnects
- Retain conservatively?    → Leaked references, no revocation
```

## 20.2 19.2 Why Reference Counting Fails Under Partition

The current `CTP_DROP` protocol is a *coordination-dependent* reference counting scheme. It has three properties that are individually reasonable but collectively problematic for partitioned operation:

1. **Reference counts are split across peers**: The exporting side tracks how many times a slot has been imported, but this count is maintained through `CTP_DROP` messages that decrement it. If messages can't flow, the count diverges from reality.

2. **GC assumes reachability information is current**: The 10-step GC sweep counts references from local pet stores, local formula dependencies, and *known* remote imports. But during partition, the "known remote imports" set is stale — the remote peer may have already dropped references, or may have created new ones via capability delegation.

3. **Revocation is a cancel + GC cycle**: When a host cancels a formula, the `withCollection` wrapper triggers GC, which should collect the cancelled formula and send `CTP_DROP` to all importers. But under partition, the drops don't arrive, and the remote peer continues holding a reference to a cancelled formula. On reconnection, the peer discovers the formula no longer exists — a dangling reference.

## 20.3 19.3 CRDT Design for Retention State

A CRDT (Conflict-Free Replicated Data Type) for retention state would allow each peer to make progress independently during partition, with convergent merge on reconnection. The natural choice is an operation-based CRDT where:

- **Retain** and **Revoke** are the two operations
- **Revoke wins** on conflict (authority-first semantics, aligned with OCap)
- Each formula's retention state includes a *vector clock* or *Lamport timestamp* so that causal ordering is preserved

```
;; --- Retention CRDT ---
;; A per-formula, per-peer retention state.

type RetentionOp
  := Retain Timestamp                    ;; "I am holding this reference"
   | Revoke Timestamp                    ;; "I revoke this capability"

type RetentionState
  :formula-id FormulaIdentifier
```

```
  :local-ops  [List RetentionOp]         ;; operations from this peer
  :remote-ops [List RetentionOp]         ;; operations from the other peer
  :merged     RetentionDecision          ;; resolved state

type RetentionDecision
  := Alive                               ;; at least one active Retain, no Revoke
   | Revoked Timestamp                   ;; authoritative revocation (wins)
   | Expired Timestamp                   ;; all Retains withdrawn, no Revoke needed

;; The merge function: join-semilattice (monotone, commutative, idempotent)
;; Revoke always wins over Retain. Later timestamps supersede earlier ones.
;; This IS a lattice: bot = no operations, top = Revoked.
```

The merge semantics are:

| Local State | Remote State | Merged Result | Rationale |
|---|---|---|---|
| Retain(t1) | Retain(t2) | Alive | Both peers agree: reference is live |
| Retain(t1) | Revoke(t2) | Revoked(t2) | Revocation wins (authority-first) |
| Revoke(t1) | Retain(t2) | Revoked(t1) | Revocation wins regardless of ordering |
| Revoke(t1) | Revoke(t2) | Revoked(max) | Both agree, use latest timestamp |
| Retain(t1) | (none) | Alive | Remote hasn't reported; assume live |
| Revoke(t1) | (none) | Revoked(t1) | Local revocation is immediately binding |

## 20.4 19.4 Session Type for CRDT-Aware Retention

```
;; --- CRDT Retention Protocol ---
;; Replaces CTP_DROP with a convergent retention protocol.

session Retention-Protocol
  ;; Phase 1: Initial capability grant
  !! Handle                                ;; export the capability
  !! RetentionEpoch                       ;; initial epoch (monotonic)
  Retention-Active

session Retention-Active
  rec Active
    &>
      ;; Normal capability usage - calls tagged with epoch
      | :use ->
          ?? [CapTP-Call Epoch]             ;; call tagged with peer's epoch
```

```
          +>
            | :epoch-current ->
                !! CapTP-Result
                Active
            | :epoch-stale ->
                !! EpochExpired            ;; clean error, not dangling ref
                Active
      ;; Local revocation (works during partition)
      | :revoke ->
          ! RevocationRecord               ;; persist CRDT op locally
          ;; Does NOT send CTP_DROP - revocation is a CRDT op
          Active                           ;; stay active; merge on reconnect
      ;; Reconnection: merge CRDT states
      | :reconnect ->
          ?? PeerRetentionState            ;; receive peer's retention ops
          ! MergedRetentionState           ;; send merge result
          ;; Apply merged revocations
          Retention-Active-Post-Merge
      ;; Clean disconnection
      | :disconnect ->
          end

session Retention-Active-Post-Merge
  rec PostMerge
    +>
      | :apply-revocation ->
          ! FormulaIdentifier              ;; revoke this formula
          ! CancelSignal                   ;; cancel its dependents
          PostMerge
      | :merge-complete ->
          Retention-Active                 ;; back to normal operation
```

## 20.5 19.5 Impact on GC

With CRDT retention, the GC algorithm gains a new input: the local retention state. The current "count references from known remote imports" step would be replaced by "consult the retention CRDT for each formula's aliveness across all peers":

```
;; --- CRDT-Aware GC ---
```

```
;; Step 4 of GC-Sweep changes: root set includes CRDT-alive formulas.

session GC-Sweep-CRDT
  ;; Steps 1-3 unchanged (assign groups, build deps, count refs)
  ! GroupAssignment
  ! [Map GroupId [Set GroupId]]
  ! [Map GroupId Nat]

  ;; Step 4 (modified): Root set includes CRDT-alive formulas
  ! [Set GroupId]                          ;; permanent roots (unchanged)
  ! [Set GroupId]                          ;; CRDT-alive: Retain without Revoke
  ! [Set GroupId]                          ;; transitively reachable from both

  ;; Steps 5-10 unchanged
  ! [Set GroupId]                          ;; groups to collect
  !! CancelBatch
  !! DropBatch
  !! DisconnectBatch
  ! DeleteBatch
  ! GCComplete
  end
```

## 20.6 19.6 Why This Matters for Revocation

In OCap security, revocation must be *authoritative*. When Alice grants Bob a capability and later wants to revoke it, the revocation must take effect regardless of Bob's state. With the current protocol, revocation requires Bob to be connected (to receive `CTP_DROP`). With a revoke-wins CRDT:

- Alice revokes locally, immediately. Her daemon marks the formula as revoked in the CRDT.
- Bob may continue using the capability during partition (his local CRDT still says "Alive").
- On reconnection, CRDT merge happens. Revoke wins. Bob discovers the formula is revoked. His calls get clean "EpochExpired" errors, not dangling references.
- Crucially: Bob cannot "un-revoke" by sending a Retain after the Revoke. The lattice is monotone — once revoked, always revoked.

This aligns with the capability discipline: the authority that grants a capability should be the authority that revokes it, without requiring consensus from the holder.

# 21 20. Practical Recommendations for the Endo Codebase

This section offers concrete, graduated recommendations for the Endo team — organized from lowest to highest implementation effort, all achievable within the existing JavaScript codebase without requiring Prologos or any external type system.

## 21.1 20.1 Immediate / Low Effort

### 21.1.1 20.1.1 Explicit State Machine for Formula Lifecycle

The formula lifecycle (§5.1) is currently implicit across `daemon.js`, `graph.js`, and `context.js`. Different code paths implement different lifecycle transitions with no single authoritative representation.

**Recommendation**: Extract a `FormulaState` enum and a `transitionState` function that validates transitions:

```
// formula-state.js
const FormulaState = /** @enum {string} */ ({
  PREFORMULATE: 'preformulate',   // ID assigned, not yet persisted
  PERSISTED: 'persisted',         // on disk, not yet in graph
  REGISTERED: 'registered',       // in graph, controller not yet created
  CONTROLLED: 'controlled',       // controller exists (synchronous)
  EVALUATING: 'evaluating',       // async evaluation in progress
  ACTIVE: 'active',               // value available
  CANCELLING: 'cancelling',       // cancel initiated, cascading
  CANCELLED: 'cancelled',         // fully cancelled
  DISPOSED: 'disposed',           // cleanup hooks run, removed from graph
});

const validTransitions = new Map([
  [FormulaState.PREFORMULATE, [FormulaState.PERSISTED]],
  [FormulaState.PERSISTED, [FormulaState.REGISTERED]],
  [FormulaState.REGISTERED, [FormulaState.CONTROLLED]],
  [FormulaState.CONTROLLED, [FormulaState.EVALUATING, FormulaState.CANCELLING]],
```

```
  [FormulaState.EVALUATING, [FormulaState.ACTIVE, FormulaState.CANCELLING]],
  [FormulaState.ACTIVE, [FormulaState.CANCELLING]],
  [FormulaState.CANCELLING, [FormulaState.CANCELLED]],
  [FormulaState.CANCELLED, [FormulaState.DISPOSED]],
]);

const transitionState = (current, next) => {
  const allowed = validTransitions.get(current);
  if (!allowed || !allowed.includes(next)) {
    throw Error(`Invalid formula transition: ${current} → ${next}`);
  }
  return next;
};
```

**Effort**: Small. The transitions already happen; this just makes them explicit and validates them. Every `Fail("unexpected formula state")` in the codebase becomes a caught invalid transition instead of a mystery crash.

**Value**: This is the JavaScript equivalent of the `Formula-Lifecycle` session type from §5. It turns implicit protocol assumptions into runtime assertions. When a new formula type is added, the valid transitions are declared alongside the type.

### 21.1.2 20.1.2 Exhaustive Formula Type Dispatch

Several dispatch sites in `daemon.js` switch on `formula.type` but do not assert exhaustiveness. TypeScript's `satisfies` or a runtime assertion can catch missing cases:

```
// At each dispatch site:
const formulaHandlers = /** @type {Record<FormulaType, Function>} */ ({
  endo: makeEndoFormula,
  host: makeHostFormula,
  guest: makeGuestFormula,
  // ... all 26+ types
});

const handler = formulaHandlers[formula.type];
if (!handler) {
  throw Error(`Unhandled formula type: ${formula.type}`);
}
```

**Effort**: Minimal per dispatch site. An afternoon of auditing to find all switch sites and add the handler map.

**Value**: This is the JavaScript equivalent of the exhaustive `&>` session type choice from §15.5. Adding a new formula type without updating *all* dispatch sites becomes an immediate, obvious error.

### 21.1.3 20.1.3 Pin Lifecycle Tracking

Add a `WeakRef` + `FinalizationRegistry` based tracker for transient pins to detect leaks:

```
const activePins = new Map();  // formulaId → { pinTime, stackTrace }

const pinTransientTracked = (formulaId) => {
  activePins.set(formulaId, {
    pinnedAt: Date.now(),
    stack: new Error('pin origin').stack,
  });
  pinTransient(formulaId);
};

const unpinTransientTracked = (formulaId) => {
  activePins.delete(formulaId);
  unpinTransient(formulaId);
};

// Periodic leak detection (e.g., every 60s)
setInterval(() => {
  const now = Date.now();
  for (const [id, info] of activePins) {
    if (now - info.pinnedAt > 30_000) {
      console.warn(`Pin leak detected: ${id}, pinned ${now - info.pinnedAt}ms ago`);
      console.warn(info.stack);
    }
  }
}, 60_000);
```

**Effort**: Small. Wraps existing pin/unpin calls with tracking.

**Value**: Detects the pin leak scenario described in §5.4 and §15.2 at runtime. Not a static guarantee, but a practical diagnostic that catches the exact failure mode.

## 21.2 20.2 Medium Effort

### 21.2.1 20.2.1 Operation Ordering Assertions

The GC protocol has critical ordering constraints (§10.3, §15.3, §15.4): pet store edge before resolution status, `CTP_DROP` before `CTP_DISCONNECT`. These can be asserted with ordered checkpoints:

```
// ordering.js - lightweight ordered-checkpoint assertions
const makeOrderedCheckpoints = (name) => {
  const reached = [];
  return harden({
    checkpoint(label) {
      reached.push(label);
    },
    assertOrder(...expectedOrder) {
      for (let i = 0; i < expectedOrder.length - 1; i++) {
        const a = reached.indexOf(expectedOrder[i]);
        const b = reached.indexOf(expectedOrder[i + 1]);
        if (a === -1 || b === -1 || a >= b) {
          throw Error(
            `${name}: ordering violation: ` +
            `expected ${expectedOrder[i]} before ${expectedOrder[i + 1]}`
          );
        }
      }
    },
  });
};

// Usage in promise resolution:
const cp = makeOrderedCheckpoints('promise-resolution');
// ... write pet store edge ...
cp.checkpoint('edge-written');
// ... write resolution status ...
cp.checkpoint('status-written');
cp.assertOrder('edge-written', 'status-written');
```

**Effort**: Medium. Requires identifying all ordering-critical code paths and inserting checkpoints. The `makeOrderedCheckpoints` utility is trivial; the audit to find all critical orderings is the real work.

**Value**: This is the JavaScript equivalent of the ordered session type `Promise-Resolution-Safe` from §15.3. It turns implicit code-ordering dependencies into explicit, runtime-checked invariants.

### 21.2.2 20.2.2 RemoteControl State Machine Hardening

The `RemoteControl` type already has an implicit state machine (Pending → Connected → Disconnected). The test suite covers ID-based connection arbitration. But there are no partition-specific tests, and the state machine doesn't model "partitioned" as a distinct state.

**Recommendation**: Add a `PARTITIONED` state and a heartbeat/timeout mechanism:

```
const RemoteControlState = {
  PENDING: 'pending',
  CONNECTED: 'connected',
  PARTITIONED: 'partitioned',   // NEW: connection silent for > timeout
  DISCONNECTED: 'disconnected',
};
```

When a connection has received no messages for a configurable timeout, transition from `CONNECTED` to `PARTITIONED`. In the `PARTITIONED` state:

- Outbound calls are queued (not dropped)
- The retention state for shared formulas is frozen (no GC of remotely-held formulas)
- A reconnection attempt is scheduled

On reconnection, transition from `PARTITIONED` back to `CONNECTED` and flush the queued calls.

**Effort**: Medium. The heartbeat mechanism needs to be added to the CapTP layer. The state machine change is straightforward given the existing `RemoteControl` structure.

**Value**: This is the architectural prerequisite for CRDT-based retention. Without a `PARTITIONED` state, the system can't distinguish "peer is slow" from "peer is unreachable," and can't defer GC decisions accordingly.

### 21.2.3 20.2.3 Structured Protocol Logging

Add structured logging at protocol boundaries that captures the session type operations (send/recv/select/offer) as named events:

```
// protocol-log.js
const protocolLog = (sessionName, step, data) => {
  console.log(JSON.stringify({
    session: sessionName,
    step: step,      // 'send', 'recv', 'select', 'offer'
    timestamp: Date.now(),
    data: typeof data === 'object' ? Object.keys(data) : typeof data,
  }));
};

// Usage:
protocolLog('CapTP-Bootstrap', 'send', { type: 'CTP_BOOTSTRAP' });
protocolLog('Formula-Lifecycle', 'transition', { from: 'CONTROLLED', to: 'EVALUATING' }
```

**Effort**: Medium (pervasive but mechanical).

**Value**: Creates an observable trace that can be compared against the session type specifications in this document. Deviations between the expected session type protocol and the actual log trace indicate protocol bugs.

## 21.3 20.3 Larger Architectural Shifts

### 21.3.1 20.3.1 CRDT Retention Layer

Implement the CRDT-based retention protocol described in §19. This is the most impactful recommendation but also the largest effort.

**Implementation sketch**:

1. Define a `RetentionCRDT` module with:
   - Per-formula, per-peer retention state
   - `retain(formulaId, timestamp)` and `revoke(formulaId, timestamp)` operations
   - `merge(localState, remoteState) → mergedState` using revoke-wins semantics
   - Persistence to disk alongside formula JSON
2. Replace `CTP_DROP` reference counting with CRDT operations:
   - When a peer imports a formula: `retain(formulaId, now)`
   - When a peer drops a reference: `revoke(formulaId, now)` (instead of `CTP_DROP`)

- On reconnection: exchange and merge retention states

3. Modify GC to consult the retention CRDT:
   - A formula with any `Alive` retention state across any peer is a root
   - A formula with `Revoked` retention from the granting host is dead regardless of remote retention

**Effort**: Substantial (multi-week, across multiple subsystems). The CRDT itself is simple (join-semilattice with revoke-wins). The integration with GC, CapTP, and persistence is the real work.

**Value**: Solves the partition-revocation problem entirely. Enables the following properties that the current system lacks:

- Revocation works during partition
- GC does not create dangling references across peers
- Reconnection is idempotent (merge is commutative and idempotent)
- No coordination required between peers for revocation

### 21.3.2 20.3.2 TypeScript Branded Types for FormulaIdentifier

The `types.d.ts` already uses branded string types (`FormulaNumber`, `NodeNumber`, `FormulaIdentifier`). Extend this pattern to enforce formula lifecycle states at the type level:

```
type UnpersistedFormulaId = string & { __brand: 'UnpersistedFormulaId' };
type PersistedFormulaId = string & { __brand: 'PersistedFormulaId' };
type ActiveFormulaId = string & { __brand: 'ActiveFormulaId' };

// Functions that require specific lifecycle states:
declare function persistFormula(id: UnpersistedFormulaId): PersistedFormulaId;
declare function evaluateFormula(id: PersistedFormulaId): ActiveFormulaId;
declare function cancelFormula(id: ActiveFormulaId): void;

// Type error: can't cancel an unpersisted formula
cancelFormula(makeFormulaId()); // Error!
```

**Effort**: Medium (requires converting JSDoc types to proper TypeScript or extending the existing branded type pattern).

**Value**: A limited form of session typing — the branded types track which lifecycle state a formula ID is in, and functions that require specific states reject IDs in the wrong state. Not as powerful as real session types, but catches a class of bugs where a formula ID is used before its formula has been properly initialized.

#### 21.3.3 20.3.3 Protocol Conformance Testing

Write integration tests that verify protocol conformance against the session type specifications in this document. For each named session type, write a test that:

1. Exercises the happy path (all protocol steps in order)
2. Exercises error branches (what happens when the protocol is violated)
3. Exercises edge cases (timeout, partition, crash-recovery)

The existing test suite has `remote-control.test.js` with state machine coverage, but no tests for partition scenarios, GC ordering, or cross-agent protocol conformance.

**Effort**: Medium-to-large (depends on how many protocols are tested).

**Value**: Even without static session typing, conformance tests verify that the implementation matches the protocol specification. When the protocol specification (this document) and the test suite agree, there is high confidence in the implementation's correctness.

## 22 21. Conclusion

The Endo system, with its formula graph, CapTP protocol, agent hierarchy, and distributed garbage collection, is a rich target for session type analysis. Every major component maps naturally to a session type:

- The *formula lifecycle* is a linear state machine with ordering constraints that session types make explicit.
- *CapTP* is a multiplexed, asynchronous protocol that maps directly to indexed async session types with duality.
- *Agent protocols* (host, guest, worker) are choice-based session types with subtyping for capability attenuation.

- *Pet store* and *mailbox* are stateful protocols with pub/sub extensions that compose with the main agent protocols.
- *Garbage collection* is an ordering-critical protocol where session types prevent the subtle bugs that arise from misordered steps.
- *Retention under partition* is a convergent state problem that CRDTs solve, with revoke-wins semantics preserving OCap revocation authority.

The analysis identifies concrete protocol properties — mutex re-entrancy prevention, pin leak detection, GC ordering safety, dispatch exhaustiveness, message number validity, and partition-safe revocation — that session types and related type-theoretic tools can verify statically. These properties are currently enforced by runtime checks, code ordering, and developer discipline. The practical recommendations in §20 show how to incrementally harden the existing codebase, from low-effort state machine extraction and exhaustive dispatch maps to medium-effort protocol ordering assertions and partition-aware state machines, culminating in a CRDT retention layer that would solve the distributed revocation problem.

Prologos's unique combination of session types, linear types, dependent types, capability annotations, and propagator-based effect ordering makes it the natural tool for this kind of protocol verification. As the language matures, it aims to provide not just a modelling language for systems like Endo, but a verification substrate where protocol correctness is a theorem, not a hope.

## 23 References


- Agoric. "Endo: Hardened JavaScript Runtime." `https://github.com/endojs/endo`. Commit `57f5ff1629e136a53fabdf0e27051b9641cd24c2`.
- Agoric. "Formula Graph Design." `packages/daemon/FORMULA-GRAPH.md` in Endo repository.
- Caires, L. and Pfenning, F. "Session Types as Intuitionistic Linear Propositions." CONCUR 2010.
- Miller, M. "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control." PhD Thesis, Johns Hopkins, 2006.

- Miller, M., Van Cutsem, T., and Tulloh, B. "Distributed Electronic Rights in JavaScript." ESOP 2013.
- Honda, K., Yoshida, N., and Carbone, M. "Multiparty Asynchronous Session Types." POPL 2008.
- Wadler, P. "Propositions as Sessions." ICFP 2012.
- Atkey, R. "Parameterised Notions of Computation." JFP 2009.
- Katsumata, S. "Parametric Effect Monads and Semantics of Effect Systems." POPL 2014.
- Prologos Project. "Protocols as Types: Composable Session Types in Prologos." `docs/tracking/principles/PROTOCOLS_AS_TYPES.org`. 2026.
- Prologos Project. "The AT Protocol as Composable Session Types." `docs/research/2026-03-05_ATP_AS_SESSION_TYPES.org`. 2026.
- Shapiro, M., Preguica, N., Baquero, C., and Zawirski, M. "Conflict-Free Replicated Data Types." SSS 2011.
- Bieniusa, A., et al. "An Optimized Conflict-Free Replicated Set." arXiv:1210.3368. 2012.
- Bocchi, L., Honda, K., and Tuosto, E. "Timed Multiparty Session Types." CONCUR 2014.
- Neykova, R., and Yoshida, N. "Session Types Go Dynamic, or How to Verify Your Python Conversations." FORTE 2017.